E n tities

C ontrollers

Services

«Entity»
User

id: Long
username: String
nam e: String
lastname: String
em ail: String
password: String
role: Role

getters/setters()

«Entity»
Study

id: Long
title: S tring
description: String
status: StudyStatus
publishStatus: PublishStatus
rejectionReason: String
visibility: Visibility
createdAt: LocalDateTime
updatedAt: LocalDateTime
endDate: LocalDateTime
studyType: StudyType
artifactCountPerTask: Integer

addParticipant(user: User)
removeParticipant(user: User)
assignQuiz(quiz: Quiz)

«Entity»
Researcher

id: Long
mainResearcher: boolean
invitedAt: LocalDateTime
acceptedAt: LocalDateTime
status: ResearcherStatus

getters/setters()

«Entity»
ResearcherPermission

id: Long
canUploadArtifacts: Boolean
canEditStudyDetails: Boolean
canInviteParticipants: Boolean
canManageResearchers: Boolean

getters/setters()

«Entity»
EvaluationTask

id: Long
questionText: String
description: String
artifactCount: int
responseFields: List<String>
correctAnswers: List<CorrectAnswerEntry>

getters/setters()

«Entity»
EvaluationCriterion

id: Long
question: String
description: String
priorityOrder: Integer
type: CriterionType

getters/setters()

«Entity»
MultipleChoiceCriterion

numberOfOptions: Integer
options: List<String>
multipleSelection: boolean

«Entity»
RatingCriterion

startValue: Integer
endValue: Integer

«Entity»
OpenEndedCriterion

minLength: Integer
maxLength: Integer

«Entity»
OpenEndedNumericCriterion

integerOnly: boolean
minValue: Double
maxValue: Double

«Entity»
CodeEditCriterion

«Entity»
ImageHighlightCriterion

numberOfAnnotations: Integer

«Entity»
ArtifactUpload

id: Long
filenam e: String
filepath: String
category: String
tags: String

getters/setters()

«Entity»
Quiz

id: Long
title: S tring
description: String
topic: String
difficulty: String

addQuestion(question: Question)
removeQuestion(question: Question)
getQuestionCount(): int
getTotalPoints(): int

«Entity»
Question

id: Long
questionText: String
points: Integer

getQuestionType(): String

«Entity»
MultipleChoiceQuestion

options: List<String>
correctAnswer: String

isCorrectAnswer(answer: String): boolean

«Entity»
OpenEndedQuestion

«Entity»
QuizResult

id: Long
answers: Map<Long, String>
pointsEarned: Map<Long, Double>
aiFeedback: Map<Long, String>
submittedAt: LocalDateTime
gradedAt: LocalDateTime
totalPointsEarned: Double
maxPossiblePoints: Integer
percentageScore: Double
status: QuizStatus

calculateTotalPoints()

«Entity»
N otification

id: Long
type: NotificationTypes
message: String
isRead: boolean
isAnswered: boolean
createdAt: LocalDateTime

getters/setters()

«Entity»
Comment

id: Long
fieldCode: String
content: String
createdAt: LocalDateTime
pinned: boolean
likeCount: int
deleted: boolean
replyCount: int

getters/setters()

«Entity»
InvitationToken

id: Long
token: String
createdAt: LocalDateTime
expiresAt: LocalDateTime

getters/setters()

«Entity»
PasswordResetToken

id: Long
token: String
expiryDate: LocalDateTime

getters/setters()

«Entity»
Report

id: Long
type: ReportType
taskId: Long
artifactId: Long
reportedUsername: String
reason: String
description: String
createdAt: LocalDateTime

getters/setters()

«Entity»
CustomCriteriaEvaluation

id: Long
fieldCode: String
value: String
createdAt: LocalDateTime

getters/setters()

«RestController»

StudyController

studyService: StudyService
jwtService: JwtService
userRepository: UserRepository
studyRepository: StudyRepository
inviteService: ParticipantInviteService

createStudy(dto: StudyRequestDTO): ResponseEntity
getAllStudies(): ResponseEntity
getStudyById(id: Long): ResponseEntity
updateStudy(id: Long, request: UpdateStudyRequestDTO): ResponseEntity
approveStudy(id: Long): ResponseEntity
rejectStudy(id: Long, reason: String): ResponseEntity
getParticipants(id: Long): ResponseEntity
inviteParticipant(id: Long, email: String): ResponseEntity

«RestController»
ResearcherController

researcherService: ResearcherService
jwtService: JwtService
userRepository: UserRepository

getResearchers(studyId: Long): ResponseEntity
inviteResearcher(studyId: Long, request: InviteResearcherRequestDTO): ResponseEntity
updatePermissions(studyId: Long, researcherId: Long, request: UpdateResearcherPermissionsDTO): ResponseEntity
removeResearcher(studyId: Long, researcherId: Long): ResponseEntity
acceptInvitation(researcherId: Long): ResponseEntity
rejectInvitation(researcherId: Long): ResponseEntity

«RestController»
EvaluationTaskController

evaluationTaskService: EvaluationTaskService
studyService: StudyService
taskArtifactService: TaskArtifactService

createTask(studyId: Long, request: EvaluationTaskCreateRequest): TaskDetailsDTO
getTasksByStudy(studyId: Long): List<TaskDetailsDTO>
getTaskById(taskId: Long): TaskDetailsDTO
updateTask(taskId: Long, request: TaskUpdateRequest): TaskDetailsDTO
updateTaskArtifacts(taskId: Long, artifactIds: List<Long>): TaskDetailsDTO

«RestController»
EvaluationCriterionController

service: EvaluationCriterionService

createCriterion(studyId: Long, request: CreateCriterionRequest): EvaluationCriterion
getByStudy(studyId: Long): List<EvaluationCriterion>
deleteCriterion(id: Long): void

«RestController»

QuizController

quizService: QuizService

createEmptyQuiz(dto: QuizCreateDTO): ResponseEntity<Quiz>
generateQuizWithAI(dto: AIQuizGenerationDTO): ResponseEntity<Quiz>
getQuizById(id: Long): ResponseEntity<Quiz>
updateQuiz(id: Long, dto: QuizUpdateDTO): ResponseEntity<Quiz>
deleteQuiz(id: Long): ResponseEntity<Void>
addMultipleChoiceQuestion(quizId: Long, dto: MultipleChoiceQuestionDTO): ResponseEntity<Quiz>
addOpenEndedQuestion(quizId: Long, dto: OpenEndedQuestionDTO): ResponseEntity<Quiz>

«RestController»

QuizAttemptController

quizResultService: QuizResultService
jwtService: JwtService

startQuiz(studyId: Long, participantId: Long): ResponseEntity
submitQuiz(quizResultId: Long, submissionDTO: QuizSubmissionDTO): ResponseEntity
getQuizResult(quizResultId: Long): ResponseEntity
acceptQuizAttempt(quizResultId: Long): ResponseEntity
rejectQuizAttempt(quizResultId: Long): ResponseEntity

«RestController»
AuthController

authService: AuthService
userService: UserService
userRepository: UserRepository

login(loginDTO: LoginDTO): ResponseEntity
register(registerDTO: RegisterDTO): ResponseEntity

«RestController»
AdminController

adminService: AdminService

getUsersFiltered(filter: UserFilter): List<UserDTO>
deleteUser(username: String): ResponseEntity
changeUserRole(username: String, newRole: String): ResponseEntity
getActionLogs(): ResponseEntity
createUser(dto: RegisterDTO): ResponseEntity
getAllStudies(): ResponseEntity
blockStudy(id: Long, reason: String): ResponseEntity

«RestController»

ArtifactUploadController

fileRepo: ArtifactUploadRepository
userRepo: UserRepository

uploadFile(file: MultipartFile): ResponseEntity
getMyArtifacts(): ResponseEntity
renameArtifact(id: Long, newName: String): ResponseEntity
deleteArtifact(id: Long): ResponseEntity
classifyArtifact(id: Long, data: Map): ResponseEntity

«RestController»
CommentController

commentService: CommentService
jwtService: JwtService
userRepository: UserRepository

createComment(req: CreateCommentRequestDTO): CommentResponseDTO
getComments(taskId: Long, fieldCode: String): List<CommentResponseDTO>
toggleLike(commentId: Long): CommentResponseDTO
deleteComment(commentId: Long): ResponseEntity
togglePin(commentId: Long): ResponseEntity

«RestController»

BulkUploadController

bulkUploadService: BulkUploadService
userRepository: UserRepository

bulkUpload(studyId: Long, zipFile: MultipartFile, csvFile: MultipartFile): ResponseEntity

«Service»

StudyService

studyRepository: StudyRepository
userRepository: UserRepository
evaluationTaskRepository: EvaluationTaskRepository
quizRepository: QuizRepository
defaultCriteriaFactory: DefaultCriteriaFactory

createStudy(study: Study, researcher: User): Study
getAllStudies(): List<Study>
getStudyById(id: Long): Study
updateStudy(id: Long, request: UpdateStudyRequestDTO, researcher: User): Study
approveStudy(studyId: Long): Study
rejectStudy(studyId: Long, reason: String): void
assignQuiz(studyId: Long, quizId: Long): Study
removeParticipantFromStudy(studyId: Long, userId: Long): void
calculateProgress(study: Study, participant: User): double

«Service»
ResearcherService

researcherRepository: ResearcherRepository
studyRepository: StudyRepository
userRepository: UserRepository

getResearchersForStudy(studyId: Long, requestingUserId: Long): List<ResearcherResponseDTO>
inviteResearcher(studyId: Long, userId: Long, username: String, ...): Researcher
updateResearcherPermissions(studyId: Long, researcherId: Long, userId: Long, request: UpdateResearcherPermissionsDTO): void
removeResearcher(studyId: Long, researcherId: Long, userId: Long): void
acceptInvitation(researcherId: Long): void
rejectInvitation(researcherId: Long): void

«Service»
EvaluationTaskService

taskRepo: EvaluationTaskRepository
artifactRepo: ArtifactUploadRepository

createTask(task: EvaluationTask): EvaluationTask
updateTask(task: EvaluationTask): EvaluationTask
getTaskById(id: Long): EvaluationTask
getTasksByStudy(study: Study): List<EvaluationTask>
updateTaskInfo(id: Long, req: TaskUpdateRequest): EvaluationTask
updateTaskArtifacts(taskId: Long, artifactIds: List<Long>): EvaluationTask
generateResponseFields(task: EvaluationTask, artifactCount: int, criterionCount: int): void
createTaskBulk(...): EvaluationTask

«Service»
EvaluationCriterionService

repository: EvaluationCriterionRepository
studyService: StudyService

createCriterion(studyId: Long, req: CreateCriterionRequest): EvaluationCriterion
getCriteriaByStudy(studyId: Long): List<EvaluationCriterion>
deleteCriterion(id: Long): void

«Service»

QuizService

quizRepository: QuizRepository
geminiQuizService: GeminiQuizService
studyRepository: StudyRepository

createQuiz(title: String, description: String, ...): Quiz
createEmptyQuiz(title: String, description: String, ...): Quiz
updateQuiz(id: Long, title: String, ...): Quiz
addMultipleChoiceQuestion(quizId: Long, questionText: String, ...): Quiz
addOpenEndedQuestion(quizId: Long, questionText: String, points: Integer): Quiz
generateQuizWithAI(title: String, description: String, ...): Quiz
deleteQuiz(quizId: Long): void

«Service»

QuizResultService

quizResultRepository: QuizResultRepository
quizRepository: QuizRepository
studyRepository: StudyRepository
userRepository: UserRepository
geminiGradingService: GeminiGradingService

createQuizResult(studyId: Long, participantId: Long): QuizResult
submitAnswers(quizResultId: Long, answers: Map<Long, String>): QuizResult
getQuizResultById(quizResultId: Long): QuizResult
acceptAttempt(quizResultId: Long): void
rejectAttempt(quizResultId: Long): void
hasParticipantSubmitted(studyId: Long, participantId: Long): boolean

«Service»

GeminiQuizService

webClient: WebClient
objectMapper: ObjectMapper
apiKey: String

generateQuestions(topic: String, numberOfQuestions: int, difficulty: String, questionType: String): List<Question>
buildPrompt(topic: String, numQuestions: int, difficulty: String, questionType: String): String
callGeminiApi(prompt: String): String
parseQuestionsFromResponse(jsonText: String, questionType: String): List<Question>

«Service»

GeminiGradingService

webClient: WebClient
objectMapper: ObjectMapper
apiKey: String

gradeOpenEndedAnswer(questionText: String, studentAnswer: String, maxPoints: Integer): GradingResult
buildGradingPrompt(question: String, answer: String, maxPoints: Integer): String
callGeminiApi(prompt: String): String
parseGradingResponse(response: String, maxPoints: Integer): GradingResult

«Service»

ParticipantInviteService

mailSender: JavaMailSender
studyRepository: StudyRepository
userRepository: UserRepository
tokenRepository: InvitationTokenRepository

sendInvitationEmail(email: String, studyId: Long): void
validateToken(tokenStr: String): InvitationToken

«Service»
AuthService

isRegisterSuccessful(registerDTO: RegisterDTO): boolean
authenticate(loginDTO: LoginDTO): boolean

«Service»
UserService

userRepository: UserRepository

registerUser(registerDTO: RegisterDTO): User
getUserByUsername(username: String): User
updateUser(username: String, updatedUser: UserDTO): boolean

«Service»
JwtService

validateAndExtractUserId(token: String): Long
generateToken(userId: Long, username: String, role: String): String

«Service»
CommentService

commentRepository: CommentRepository
evaluationTaskRepository: EvaluationTaskRepository
userRepository: UserRepository

createComment(req: CreateCommentRequestDTO, userId: Long): CommentResponseDTO
getCommentsForField(taskId: Long, fieldCode: String): List<CommentResponseDTO>
toggleLike(commentId: Long, userId: Long): CommentResponseDTO
deleteComment(commentId: Long, deleter: User): void
togglePin(commentId: Long, actor: User): CommentResponseDTO

«Service»

BulkUploadService

studyRepository: StudyRepository
artifactRepo: ArtifactUploadRepository
taskService: EvaluationTaskService

processBulkUpload(studyId: Long, zipFile: MultipartFile, csvFile: MultipartFile, researcher: User): void

«Service»

DefaultCriteriaFactory

criterionRepo: EvaluationCriterionRepository

createDefaultsForStudy(study: Study): void
createBugDefaults(study: Study): void
createCloneDefaults(study: Study): void
createSnapshotDefaults(study: Study): void
createSolidDefaults(study: Study): void

«Service»
PasswordResetService

userRepository: UserRepository
tokenRepository: PasswordResetTokenRepository
mailSender: JavaMailSender

sendResetLink(email: String): void
resetPassword(dto: ResetPasswordForgotDTO): void

«Service»
AdminService

userRepository: UserRepository
studyRepository: StudyRepository
adminActionLogRepository: AdminActionLogRepository

getAllUsersFiltered(filter: UserFilter): List<UserDTO>
deleteUserByUsername(username: String): boolean
changeUserRole(username: String, newRole: String): boolean
getAllActionLogs(): List<AdminActionLogDTO>
createNewUser(dto: RegisterDTO): boolean
getAllStudies(): List<StudyResponseDTO>
blockStudy(id: Long, reason: String): boolean

«Service»
TaskArtifactService

repo: ArtifactUploadRepository

getArtifactsByIds(ids: List<Long>): List<ArtifactUpload>

«Service»
CustomCriteriaEvaluationService

evaluationRepository: CustomCriteriaEvaluationRepository
taskRepository: EvaluationTaskRepository
criterionRepository: EvaluationCriterionRepository
userRepository: UserRepository

processSubmitRequest(request: CustomCriteriaSubmitRequestDTO, participant: User): void

«Service»
EvaluationService

evaluationTaskRepository: EvaluationTaskRepository

submitEvaluation(req: EvaluationRequestDTO, userId: Long): void

researcher

1

*

participants

*

*

reviewers

*

*

uploads

1

*

receives

1

*

creates

1

*

author

*

1

attem pts

1

*

participant

*

1

invited as

1

* *

1
1

1

contains

1

**

1

defines

1

* *

1

has co-researchers

1

* *

has

1

0..1
0..1

1

1

* *

1

1

*

has

1

1

evaluates

*

*

has discussions

1

*

*

1

com pleted by

*

*

contains

1

* *

1

generates

1

*
*

1

*

1

replies

1

*

participant

*

1

*

1

*

1

participant

*

1

reporter

*

1

uses

uses uses uses uses

uses uses

uses uses uses uses uses uses uses uses uses uses uses

uses uses

uses

uses uses